

Week01 | 作业 | 把 Demo 思维收口为工程蓝图

问题定义、系统蓝图、Done 标准与风险边界

本页结构

本页目标	1
你本周到底要交什么	1
问题定义	1
系统蓝图	1
Done 标准	1
风险边界	2
为什么这份作业重要	2
本页与实验页的边界	2
4 个必交工件	2
A. Problem Framing Card 问题定义卡	2
B. System Blueprint v0.1 系统蓝图	3
C. DoD v0.1 Definition of Done	3
D. Boundary Checklist v0.1 风险边界清单	3
推荐作答路径	3
系统蓝图范式示意	4
评分 Rubric	4
高分作业长什么样	5
提交要求	5
提交前自检清单	5
课时与实验回看入口	5

本页目标

这份作业不是让你写一个“AI 想法”，而是让你交出一版可评审、可接续、可落地的系统蓝图雏形。

Week01 的重点，不是证明你能把功能讲得多酷，而是证明你已经能把一个 AI 项目放回真实业务、真实边界和真实交付里讨论。

你本周到底要交什么

问题定义

先讲清楚这到底是一个什么业务问题，而不是什么 AI 功能创意。

你要写清使用者是谁、旧流程为什么不够、真正的业务结果是什么，以及什么不在本周目标内。

系统蓝图

给出一版最小系统蓝图。

它至少要包含输入、处理、服务、动作和控制五部分，不能只画“用户 – LLM – 向量库”。

Done 标准

写出本周的 Done v0.1。

不是“更智能”“更准确”，而是能不能判断、能不能验收、能不能说明为什么算完成。

风险边界

从第一周开始把 PII、权限、动作边界、人工介入、失败回退和待确认事项写清楚。
这不是附录，而是系统能不能继续往下做的前提。

! 一页记住

Week01 的作业不是“交一个想法”，而是第一次把 Demo 思维收口成工程口径。
你交的这四份工件，后面会一路接到 Week02 的数据契约、Week03 的采集与入湖、Week08 的 RAG API、Week10 的工具层、Week12 的 tracing 和 Week14 的版本治理。

为什么这份作业重要

如果这一页只停留在“我想做一个 AI 助手”，后面每一周都会越学越散。
真正有价值的起点，不是把工具栈写满，而是先把问题定义、系统边界、Done 标准和风险边界立住。
下面这张表，说明这四份工件为什么不是孤立文档，而是后续周次的前置输入：

后续周次	会接走什么	这份作业现在先要交什么
Week02	输入边界、数据契约	业务输入范围、PII、权限、关键字段
Week03	采集与入湖	输入源、原始数据形态、处理链起点
Week08	RAG API 与证据链	系统蓝图里的检索与服务层
Week10	工具层与 HITL	可执行 / 不可执行动作、人工接管
Week12	tracing / 观测	Done 里的可观测、可定位、可追溯
Week14	治理与发布	风险边界、回退、版本与责任边界

本页与实验页的边界

这一周既有实验，也有作业，但两者不是一回事。

页面	关注点	你要产出的东西
实验页	把项目最小基线跑起来	环境、目录、最小运行链路
作业页	把系统口径讲清楚	问题定义、蓝图、Done、风险边界

所以，实验页更偏“把起点搭起来”，作业页更偏“把系统说清楚”。
不要把实验截图当作作业正文，也不要把作业写成项目使用说明。

4 个必交工件

A. Problem Framing Card | 问题定义卡

- 为什么要交：先把“解决什么问题”讲清楚，避免一开始就把系统写成万能助手。
- 要回答什么：
 - 主要用户是谁
 - 当前流程哪里卡住
 - 成功的业务结果是什么
 - 哪些内容本周不做

- 最低要求：能区分“业务问题”与“AI 功能想象”
- 高分表现：问题定义清楚、边界清楚、非目标写清楚
- 常见误区：把功能列表当问题定义，把愿景口号当交付目标

B. System Blueprint v0.1 | 系统蓝图

- 为什么要交：系统蓝图决定你后面是在补链路，还是在堆功能。
- 要回答什么：
 - 输入从哪里来
 - 经过哪些处理层
 - 如何形成检索/服务能力
 - 哪些动作可以执行
 - 哪些控制面必须始终在线
- 最低要求：至少画出输入、处理、服务、动作、控制五部分
- 高分表现：同时体现数据路径与控制面，不把系统画成三段式 Demo
- 常见误区：只有 LLM 和向量库，没有治理、权限、边界和动作层

C. DoD v0.1 | Definition of Done

- 为什么要交：Done 是交付标准，不是功能愿望。
- 要回答什么：
 - 功能 Done：本周最小功能是否成立
 - 工程 Done：是否可追溯、可观测、可回归、可回滚
 - 风险 Done：是否有边界、人工介入和失败回退
- 最低要求：每条 Done 都能判断，不是模糊口号
- 高分表现：目标、验收口径、证据/检查方式能一一对应
- 常见误区：写“更智能”“更稳定”“更准确”这种无法验收的话

D. Boundary Checklist v0.1 | 风险边界清单

- 为什么要交：边界不是提醒语，而是系统必须服从的硬约束。
- 要回答什么：
 - PII 怎么分级
 - 权限边界到哪里
 - 哪些动作能做 / 不能做
 - 什么情况下必须转人工
 - 失败后怎么处理
 - 哪些问题还需要业务确认
- 最低要求：能落到具体条件，不是泛泛提醒
- 高分表现：边界一旦触发，就知道系统应该怎样处理
- 常见误区：只写风险名词，不写触发条件和处理动作

推荐作答路径

建议按这个顺序完成，不要一上来就画图：

1. 回看课时 1 和课时 2，把 Demo 与生产、Done 与边界的判断先立住。
2. 先写问题定义卡，尤其先写清非目标。
3. 再画系统蓝图，只画最小闭环，不要追求一口气画全。
4. 然后写 DoD，把“完成”写成可判断的口径。

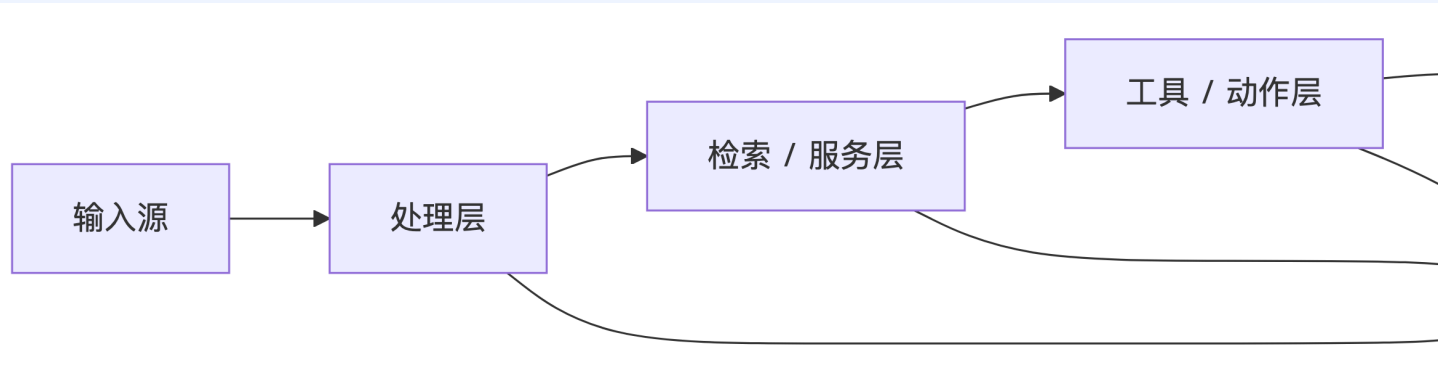
5. 最后反推风险边界：PII、权限、动作和 HITL 分别在哪里。

💡 Tip

先写非目标，再写目标。
很多作业之所以越写越大，不是因为系统真的需要那么复杂，而是一开始没有把“不做什么”写下来。

系统蓝图范式示意

下面这张图只给你一个最小范式，用来提醒蓝图至少要讲清哪些层。
它不是标准答案，更不是让你照抄的模板。



i 使用方式

- 你可以参考这个结构，但不要照抄节点名称
- 真正要交的是你自己的系统理解，而不是一张标准示意图
- 这张图的价值，在于提醒你：数据路径和控制面必须同时出现

评分 Rubric

维度	分值	高分表现	中档表现	低分表现
问题定义	20	业务问题、用户、结果、非目标都清楚	能看出场景，但边界不清	只有功能愿景，没有问题定义
系统蓝图	25	结构完整，既有数据路径也有控制面	有主链，但层次不全	只画成 Demo 三段式
DoD 标准	25	可判断、可验收、可追溯	有标准，但口径偏模糊	多数表述不可验收
风险边界	20	PII、权限、动作、HITL 写清	写到了风险，但不够具体	只有名词，没有处理规则
可接续性	10	能明显接到后续周次	隐约能接	本周内容与后续脱节

高分作业长什么样

💡 高分作业的共同特征

- 问题定义是业务语言，不是功能宣传语
- 蓝图同时表达了数据路径和控制面
- Done 可以被检查，而不是靠感觉判断
- 风险边界能触发具体动作
- 能看出来这些工件如何接入后续周次

⚠️ 典型失分模式

- 把“AI 助手”当问题定义
- 只画用户、模型、向量库三段式
- Done 写成“更准确、更智能”
- 风险边界只有名词，没有规则
- 页面像项目宣传稿，而不是工程交付页

提交要求

- 统一提交一份文档，格式可为 Markdown、PDF 或 Word
- 文档内必须按 4 个工件分节
- 图和表必须能独立阅读，不依赖口头补充
- 命名建议：`week01-assignment-姓名或组名`

提交前自检清单

- ☐ 我写的是业务问题，不是 AI 功能愿景
- ☐ 我写清了主要用户、成功结果和非目标
- ☐ 我的蓝图不只是“用户 – LLM – 向量库”
- ☐ 我的蓝图里能看到输入、处理、服务、动作、控制
- ☐ 我的 Done 是可判断、可验收的
- ☐ 我写清了 PII 或敏感数据边界
- ☐ 我写清了权限边界
- ☐ 我写清了哪些动作能做、哪些动作不能做
- ☐ 我写清了 HITL 触发条件
- ☐ 我能解释这份作业如何接到后续周次

课时与实验回看入口

如果你写到一半卡住，建议回看这些页面：

- [课时 1：为什么你的 AI Demo 一上生产就翻车？](#)
- [课时 2：知识库问答 + 工单联动怎么定义 Done](#)
- [课时 3：企业级 AI 到底该走哪条数据工程路线](#)
- [课时 4：工程底座、风险边界与蓝图怎么一次定清](#)
- [实验页：项目基线与系统蓝图初始化](#)